Lab 1 Report

Rasul Abbaszada

April 12, 2026

1 Part A: FFT Basics

In this section, I explored the fundamentals of the Fast Fourier Transform (FFT) by analyzing a sinusoidal signal. I generated a test signal and computed its FFT to observe its frequency components in the complex plane.

The real part of the FFT represents the cosine components (even parts) of the signal, while the imaginary part represents the sine components (odd parts). Together, these describe both the amplitude and the phase of the frequencies present. When I plotted the FFT of the real-valued signal, I observed a symmetric pattern around 0 Hz. This is because real-valued signals possess conjugate symmetry in the frequency domain, meaning the magnitude at a positive frequency is identical to the magnitude at its negative counterpart.

In the resulting plots, I identified main peaks at ± 100 Hz, which matches the input frequency of the generated sine wave. When plotting the magnitude spectrum, I scaled the results by $2/N$. The division by N normalizes the FFT sum into an average amplitude, while the factor of 2 is necessary for a one-sided plot to recover the full physical amplitude, as the energy is originally split between positive and negative frequencies.

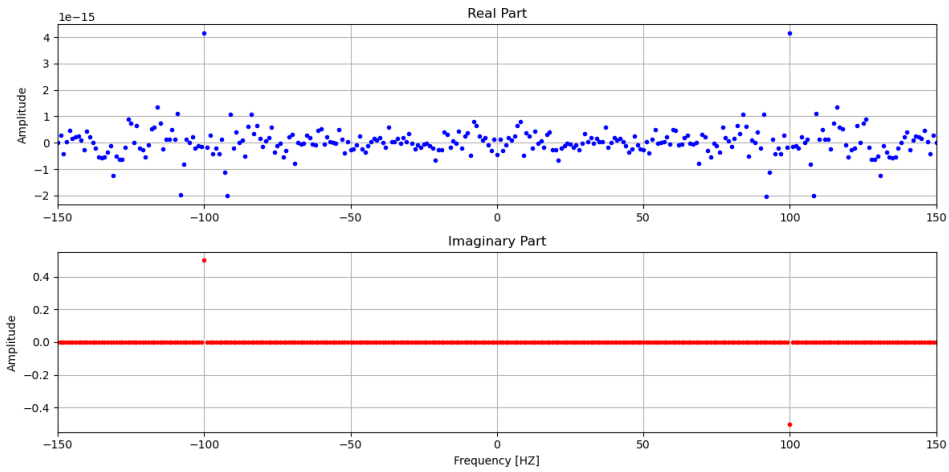

Figure 1: FFT Real and Imaginary parts vs. Frequency.

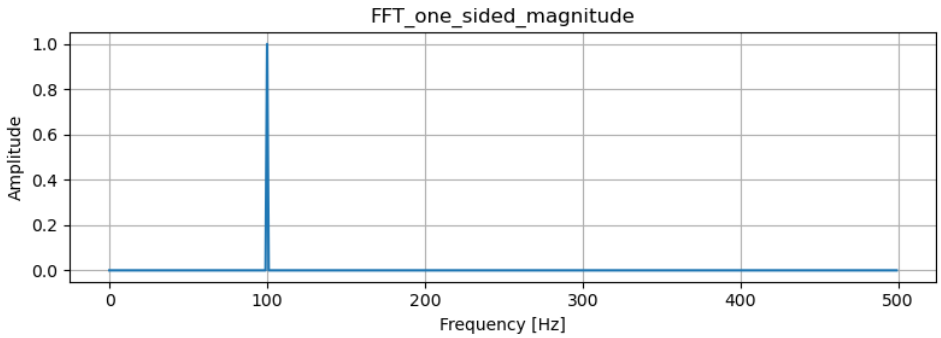

Figure 2: One-sided magnitude spectrum.

2 Part B: Manual DFT vs. FFT

I implemented a Discrete Fourier Transform (DFT) manually using a transformation matrix to compare its performance with the optimized FFT algorithm.

Mathematically, the manual DFT has a computational complexity of $O(N^2)$, meaning the work increases with the square of the signal length. The FFT reduces this to $O(N \log N)$, which is much more efficient for large datasets. The manual DFT and the FFT produced identical magnitude spectra, verifying that the FFT is an optimized implementation of the same mathematical transformation.

The transformation matrix W can be interpreted as a set of complex basis sinusoids used to analyze the signal. Each row of the matrix corresponds to a specific frequency, and performing matrix multiplication is effectively a process of comparing the input signal against each of these basis waves. In the heatmaps of the real and imaginary parts of W, the color patterns represent the oscillating values of these sinusoids. As the row index increases, the periodic oscillations in the heatmap become more frequent. This is because higher rows correspond to higher frequencies, completing more cycles across the same number of time samples.

The resolution and spacing of the frequency bins are determined by the relationship between the sampling rate (F_s) and the total number of samples (N). The distance between frequency bins is defined as $\Delta f = F_s/N$. Consequently, increasing the number of samples while keeping the sampling rate constant increases the total recording time, which in turn decreases the distance between bins. This results in a higher frequency resolution.

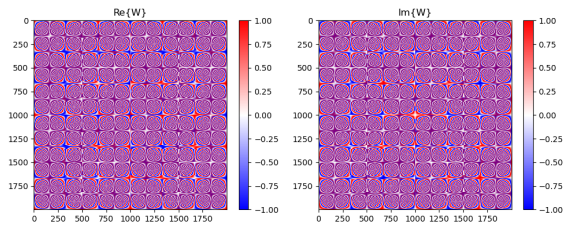

Figure 3: Heatmaps of the Real and Imaginary parts of the DFT matrix W.

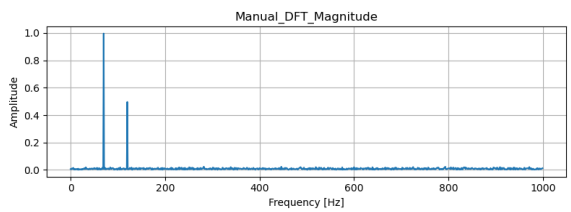

Figure 4: Manual DFT Magnitude

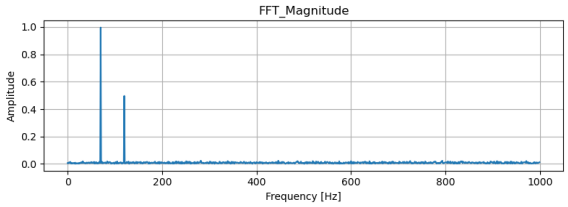

Figure 5: FFT Magnitude

3 Part C: FFT-Based Notch Filtering

In this section, I implemented a frequency-domain notch filter to remove a specific 600 Hz interference signal. By transforming the signal into the frequency domain, I could target the noise with high precision. Applying a cutoff or a specific filter width effectively creates a "hole" in the signal's spectrum. In a typical low-pass filter, a cutoff frequency f_{cut} removes all energy above that point, whereas in this notch filter, it defines the boundaries around 600 Hz where the signal is suppressed. This process directly results in the suppression of bandpower, which we measure in decibels (dB). The dB value represents the logarithmic ratio of the signal's power after filtering compared to its original state; a significantly negative value indicates that the unwanted noise energy has been successfully eliminated from the spectrum.

The precision of this filter is fundamentally limited by the frequency bin spacing, defined as $\Delta f = F_s/N$. This spacing acts like a grid; if the interference frequency does not align perfectly with one of these bins, the filter cannot target it with absolute accuracy. This misalignment leads to a phenomenon known as spectral leakage, where the energy of the interference spreads across multiple neighboring bins rather than staying in one spot. This leakage occurs because the signal is "cut off" abruptly at the edges of the FFT window.

To combat this, windowing functions such as a Hann or Hamming window can be used to taper the signal edges smoothly. This smoothing reduces the spreading of energy, confining the noise to a much narrower frequency range. Consequently, using a window allows for a more effective notch design, as it prevents the interference from "leaking" into the frequencies we want to keep, making the final denoised signal much cleaner and more accurate to the original.

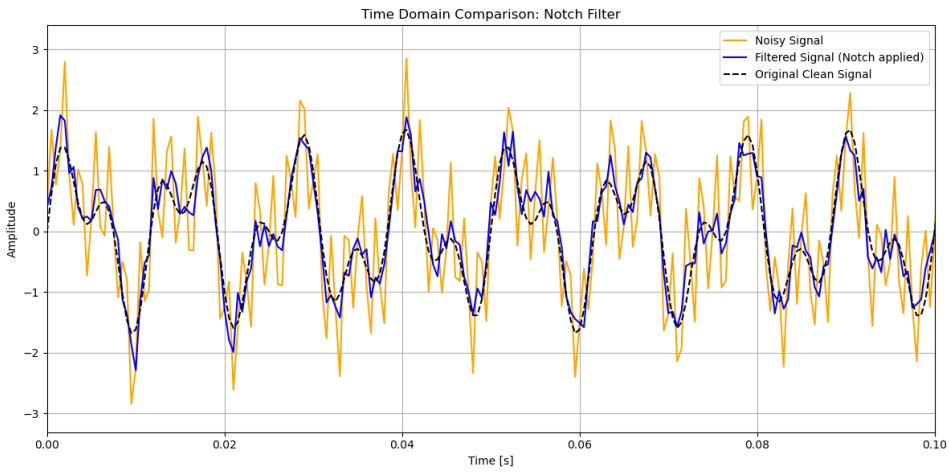

Figure 6: Time-domain comparison showing the noisy signal, the cleaned version, and filtered signal

4 Part D: Derivatives via FFT

In this section, I compared the accuracy of calculating signal derivatives using the standard finite-difference method versus the spectral FFT method. Upon inspecting the plotted results, it is clear that the FFT spectral derivative provides a much closer match to the true analytical derivative than the finite difference approach. This superior accuracy becomes even more apparent as the number of grid points N is increased. While the finite difference method continues to show a measurable error due to its approximate nature, the FFT method converges much more rapidly toward the exact analytical values, often reaching near-perfect accuracy for smooth functions.

The difference in performance is due to the fundamental mathematical approach of each method. The finite difference method is a local approximation based on the distance between neighboring data points. This approach only considers the immediate slope between points, which introduces truncation error and makes it sensitive to the grid spacing. In contrast, the FFT method is a global approach that represents the entire signal as a combination of complex sinusoids. By transforming the signal into the frequency domain, the derivative is calculated analytically by multiplying the Fourier components by $i2\pi f$. By using the global information of the entire signal, it avoids local approximation errors.

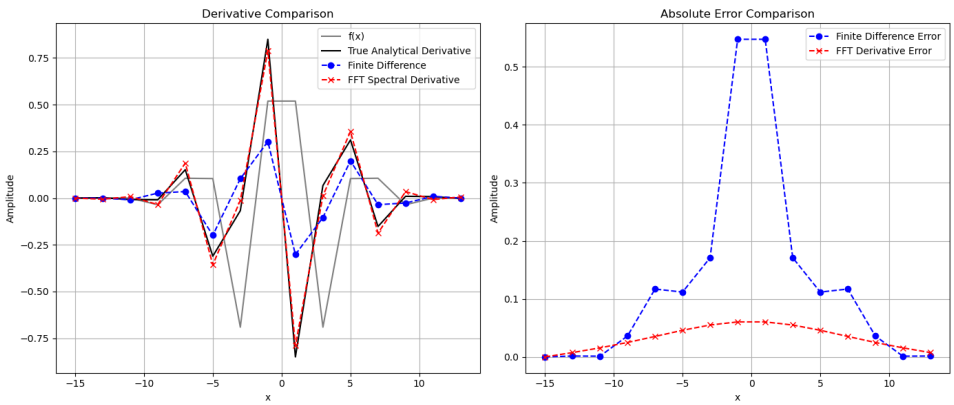

Figure 7: Comparison of the True derivative, Finite-Difference derivative, and FFT derivative.